

Guião 11 – NFS, iSCSI e Object Storage com MinIO

Infraestruturas de Computação na Nuvem, Aula 11

1 Objetivos

- Observar os tipos de storage de rede suportados pelo Proxmox e a sua relação com a migração ao vivo.
- Configurar um servidor NFS numa VM e montar a partilha a partir de uma VM cliente.
- Configurar um alvo (target) iSCSI básico e ligar-se a partir de um initiator noutra VM.
- Demonstrar a diferença essencial entre acesso ao nível de ficheiro e ao nível de bloco, com acesso concorrente.
- Correr o MinIO e utilizar o cliente mc para criar um bucket e enviar/obter um objeto.
- Usar o NFS como PersistentVolume no cluster Kubernetes (opcional).

2 Pré-requisitos e Material

- Duas VMs Linux no Proxmox: uma para servidor (**servidor**) e outra para cliente (**cliente**), na mesma rede. Pode reutilizar VMs de aulas anteriores.
- Acesso à interface web do Proxmox, para a Parte A.
- Docker instalado na VM onde se vai correr o MinIO (pode ser a mesma VM servidor).
- Acesso com privilégios de administração (**sudo**) em ambas as VMs.
- Opcionalmente, o cluster Kubernetes do Guião 05, para a Parte E.
- Pacotes necessários:

```
# Na VM servidor
$ sudo apt update
$ sudo apt install -y nfs-kernel-server targetcli-fb

# Na VM cliente
$ sudo apt update
$ sudo apt install -y nfs-common open-iscsi
```

3 Introdução

Este guião cobre três formas distintas de armazenamento em rede, correspondentes aos conceitos vistos na teoria: NFS (ao nível de ficheiro, tipo NAS), iSCSI (ao nível de bloco, tipo SAN) e object storage (via MinIO, compatível com a API S3).

O objetivo não é apenas configurar cada um, mas perceber *porque* diferem. O exercício mais importante do guião está na Parte C: montar a mesma partilha NFS em dois clientes ao mesmo tempo funciona; fazer o equivalente com iSCSI é uma receita para corromper dados. Compreender esta diferença é compreender a distinção entre ficheiro e bloco.

4 Parte A – Armazenamento de rede no Proxmox

1. **Observar os tipos de storage suportados.** Na interface web, acesse a *Datacenter* → *Storage* → *Add* e consulte (sem adicionar nada) a lista de tipos disponíveis. Identifique os que correspondem a armazenamento de rede: *NFS*, *iSCSI*, *CIFS/SMB*, *Ceph RBD*, entre outros.
2. **Relacionar com a migração ao vivo.** Consulte os storages atualmente configurados e a coluna que indica se são partilhados (*Shared*):

```
$ pvesm status
```

Responda, para registo: se o disco de uma VM estiver num storage local, o que teria de acontecer para a migrar para outro nó? E se estivesse num storage partilhado por NFS ou Ceph? (Retome o conceito de migração ao vivo da Aula 03.)

5 Parte B – Partilha de Ficheiros com NFS

1. Na VM **servidor**, criar a pasta a partilhar e definir permissões:

```
$ sudo mkdir -p /srv/nfs/partilha
$ sudo chown nobody:nogroup /srv/nfs/partilha
```

2. Editar `/etc/exports` para exportar a pasta para a rede do cliente (substituir pelo IP/sub-rede real):

```
/srv/nfs/partilha 192.168.56.0/24(rw,sync,no_subtree_check)
```

3. Aplicar a exportação e reiniciar o serviço:

```
$ sudo exportfs -ra
$ sudo systemctl restart nfs-kernel-server
```

4. Na VM **cliente**, criar o ponto de montagem e montar a partilha:

```
$ sudo mkdir -p /mnt/nfs
$ sudo mount servidor:/srv/nfs/partilha /mnt/nfs
$ df -h /mnt/nfs
```

5. Testar a escrita a partir do cliente e confirmar visibilidade no servidor:

```
$ echo "ola via nfs" | sudo tee /mnt/nfs/teste.txt
# no servidor:
$ cat /srv/nfs/partilha/teste.txt
```

6. **Testar o acesso concorrente.** Monte a *mesma* partilha também no servidor (ou numa terceira VM, se disponível) e escreva a partir dos dois lados em simultâneo:

```
# no cliente:
$ echo "escrito pelo cliente" | sudo tee /mnt/nfs/concorrente.txt
# no servidor (ou noutra cliente), imediatamente a seguir:
$ cat /srv/nfs/partilha/concorrente.txt
$ echo "acrescentado pelo servidor" | sudo tee -a /srv/nfs/partilha/concorrente.
txt
# de volta ao cliente:
$ cat /mnt/nfs/concorrente.txt
```

Confirme que ambos os lados veem as alterações um do outro. É o servidor NFS que gere o sistema de ficheiros e coordena os acessos – guarde esta observação para comparar com o iSCSI.

6 Parte C – Armazenamento em Bloco com iSCSI

1. Na VM **servidor**, criar um ficheiro de 500 MB para servir de disco virtual do target iSCSI:

```
$ sudo mkdir -p /srv/iscsi
$ sudo truncate -s 500M /srv/iscsi/disco0.img
```

2. Configurar o target iSCSI com **targetcli**:

```
$ sudo targetcli
/> backstores/fileio create disco0 /srv/iscsi/disco0.img
/> iscsi/ create iqn.2026-11.icn.estga:servidor
/> iscsi/iqn.2026-11.icn.estga:servidor/tpg1/luns create /backstores/fileio/
    disco0
/> iscsi/iqn.2026-11.icn.estga:servidor/tpg1/acls create iqn.2026-11.icn.estga:
    cliente
/> saveconfig
/> exit
```

3. Na VM **cliente**, definir o próprio IQN (deve corresponder ao ACL criado acima), em **/etc/iscsi/initiatorname.iscsi**:

```
InitiatorName=iqn.2026-11.icn.estga:cliente
```

4. Descobrir e ligar-se ao target a partir do cliente:

```
$ sudo systemctl restart open-iscsi
$ sudo iscsiadm -m discovery -t sendtargets -p servidor
$ sudo iscsiadm -m node --targetname iqn.2026-11.icn.estga:servidor --login
```

5. Confirmar que o novo disco de bloco apareceu e formatá-lo/montá-lo:

```
$ lsblk
$ sudo mkfs.ext4 /dev/sdX # substituir sdX pelo disco iSCSI identificado
$ sudo mkdir -p /mnt/iscsi
$ sudo mount /dev/sdX /mnt/iscsi
```

6. **Observar a diferença essencial face ao NFS.** Repare no que acabou de fazer: teve de *formatar* o dispositivo. No NFS isso nunca foi necessário, porque o sistema de ficheiros já existia do lado do servidor.

Escreva um ficheiro e confirme:

```
$ echo "ola via iscsi" | sudo tee /mnt/iscsi/teste.txt
$ ls -l /mnt/iscsi
```

Não faça isto: montar este mesmo LUN em escrita a partir de um segundo cliente ao mesmo tempo. Cada cliente teria a sua própria cache e estruturas do sistema de ficheiros, sem saber da existência do outro, e o resultado seria a corrupção do ext4. É esta a diferença fundamental face ao NFS: no iSCSI o *cliente* gere o sistema de ficheiros, e não há ninguém a coordenar acessos concorrentes.

Para permitir escrita simultânea sobre um dispositivo de bloco partilhado seria necessário um sistema de ficheiros de cluster (ex. GFS2, OCFS2), com bloqueio distribuído.

7 Parte D – Object Storage com MinIO

1. Correr o MinIO num container Docker:

```
$ docker run -d --name minio -p 9000:9000 -p 9001:9001 \
  -e "MINIO_ROOT_USER=admin" -e "MINIO_ROOT_PASSWORD=minioadmin123" \
  -v /srv/minio-data:/data \
  minio/minio server /data --console-address ":9001"
```

2. Instalar o cliente mc e configurar o alias para o servidor local:

```
$ curl -O https://dl.min.io/client/mc/release/linux-amd64/mc
$ chmod +x mc
$ ./mc alias set localminio http://localhost:9000 admin minioadmin123
```

3. Criar um bucket:

```
$ ./mc mb localminio/dados-aula11
```

4. Enviar (upload) um objeto para o bucket:

```
$ echo "objeto de teste" > objeto.txt
$ ./mc cp objeto.txt localminio/dados-aula11/objeto.txt
```

5. Listar o conteúdo do bucket e obter (download) o objeto de volta:

```
$ ./mc ls localminio/dados-aula11
$ ./mc cp localminio/dados-aula11/objeto.txt objeto-descarregado.txt
$ cat objeto-descarregado.txt
```

8 Parte E – NFS como PersistentVolume no Kubernetes (opcional)

Esta parte assume o cluster do Guião 05 e reutiliza o servidor NFS configurado na Parte B.

1. Instalar o cliente NFS em todos os nós do cluster:

```
$ sudo apt install -y nfs-common
```

2. Criar o PersistentVolume e o PersistentVolumeClaim. Crie nfs-pv.yaml (substitua o IP do servidor):

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfs-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteMany
  nfs:
    server: <IP-do-servidor-NFS>
    path: /srv/nfs/partilha
  persistentVolumeReclaimPolicy: Retain
---
```

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nfs-pvc
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 1Gi
  storageClassName: ""

```

Repare no modo de acesso `ReadWriteMany`: só é possível porque o NFS é armazenamento ao nível de ficheiro.

3. Aplicar e verificar a ligação (*bound*):

```

$ kubectl apply -f nfs-pv.yaml
$ kubectl get pv,pvc

```

4. Montar o volume em dois Pods em nós diferentes e confirmar que ambos veem os mesmos ficheiros:

```

$ kubectl run escritor --image=busybox:1.36 --restart=Never \
  --overrides='{ "spec": { "containers": [ { "name": "c", "image": "busybox:1.36", "
    command": [ "sh", "-c", "echo escrito-pelo-pod > /dados/k8s.txt; sleep
    3600" ], "volumeMounts": [ { "name": "v", "mountPath": "/dados" } ] }, { "
    name": "v", "persistentVolumeClaim": { "claimName": "nfs-pvc" } } ] } }'
$ kubectl run leitor --image=busybox:1.36 --restart=Never \
  --overrides='{ "spec": { "containers": [ { "name": "c", "image": "busybox:1.36", "
    command": [ "sh", "-c", "sleep 3600" ], "volumeMounts": [ { "name": "v", "mountPath
    ": "/dados" } ] }, { "name": "v", "persistentVolumeClaim": { "claimName
    ": "nfs-pvc" } } ] } }'
$ kubectl exec leitor -- cat /dados/k8s.txt

```

5. Confirmar no servidor NFS que o ficheiro criado pelo Pod está lá:

```

$ cat /srv/nfs/partilha/k8s.txt

```

6. Limpeza:

```

$ kubectl delete pod escritor leitor
$ kubectl delete -f nfs-pv.yaml

```

9 Entregável / Questões de Verificação

- Capturas de ecrã (ou output copiado) de: montagem NFS bem-sucedida no cliente, `lsblk` com o disco iSCSI montado, e `mc ls` a mostrar o objeto no bucket.
- No caso do NFS, o cliente acede aos dados ao nível de ficheiro ou de bloco? E no caso do iSCSI? Justifique com base no que foi montado em cada caso.
- Descreva o que observou no teste de acesso concorrente ao NFS (Parte B) e a razão pela qual o mesmo não deve ser feito com o volume iSCSI. Que componente, em cada caso, é responsável por gerir o sistema de ficheiros?

- d) Que vantagem tem o MinIO/S3 sobre o NFS quando se trata de armazenar milhões de ficheiros pequenos e não estruturados?
- e) O acesso ao MinIO foi feito através de que tipo de API? Compare com o modelo de acesso usado no NFS e no iSCSI.
- f) Se o disco de uma VM estiver num storage local do Proxmox, o que é necessário para a migrar para outro nó? E se estiver num storage partilhado? Relacione com a migração ao vivo (Aula 03).
- g) (Se fez a Parte E) Porque é que o PersistentVolume NFS pôde ser declarado com **ReadWriteMany**? Que modo de acesso teria de usar se o volume fosse iSCSI, e porquê?
- h) Indique, para cada um dos três mecanismos usados (NFS, iSCSI, MinIO), um cenário real de utilização em ambiente de cloud.